

Architecture: backends, credentials, projects, and model knowledge

This tool starts AI agents against a churning set of LLM endpoints — vLLM instances administered by other people, Bedrock through two accounts with Gov and non-Gov keys, home servers, hosted APIs — and must pick the right subset (and the right money) per project. Most ad-hoc setups conflate several kinds of state into one script or one .env file. This note separates them into components with different owners, different lifetimes, and different sharing rules, then shows the one-directional pipeline that composes them at launch.

The shape

SHARED, IN GIT (no secrets, ever)

backends.yaml	projects.yaml	models.yaml	catalog.json
what exists,	what THIS	what we	vendored
how to reach	project may	believe about	price/context
it, which	use (the	models the	tables for
credential	right money)	endpoints	hosted APIs
NAME it needs		won't explain	

PRIVATE, PER MACHINE

credential store	machine.yaml
name → secret,	local quirks:
0600, never git	addresses, ports

LAUNCH PIPELINE (runs when an agent starts, or as `ma models`)
select → resolve credentials → probe endpoints → diff vs
last-seen → merge model facts → render config → start proxy
→ start agent

DERIVED / OBSERVED, PER MACHINE

last-seen snapshot	what the probes saw last time -
(~/.local/state/...)	memory for change reports only,
	never an input to routing

A one-line summary of each component's epistemic role: the registry and project policy are **intentions**, the credential store is **capability**, models.yaml is **belief**, probes are **observation**, and the snapshot is **memory of observations**. Availability is always derived — registry ∩ credentials-present ∩ endpoint-answering — never stored as fact.

Component	Question it answers	Lifetime
Backend registry	What endpoints exist, and how do you talk to them?	Months
Credential store	What secrets do <i>I</i> hold, on <i>this</i> machine, <i>today</i> ?	Hours–months
Project policy	Which backends/models may <i>this project</i> use?	Weeks
Model knowledge	What do we believe about model X that no endpoint will tell us?	Weeks; corrected as learned
Price catalog	What does a token cost on a hosted API?	External; vendored copy updated
Machine config	How does <i>this</i> machine reach things?	Stable
Last-seen snapshot	What did probes observe last launch?	Per launch

The link between shared and private is **indirection by name**: the registry says a backend needs a credential *called* `bedrock-projA-gov`; the credential store maps that name to a value. A coworker who clones the registry maps the same name to *their* key. Nothing shared ever contains a secret, and nothing secret ever needs to know the registry's shape.

backends.yaml — connectivity, shared, in git

Deliberately narrow: how to reach an endpoint and what credential name it needs. Facts about the *models* an endpoint serves belong in `models.yaml`, so this file stays pure connectivity and changes only when infrastructure does.

```
backends:
  work-vllm-3:
    type: openai-compatible      # vLLM speaks /v1/models and /v1/chat/completions
    api_base: http://vllm-3.internal:8000
    discovery: dynamic           # probe at launch; admins change the served model
    credential: none

  home-vllm:
    type: openai-compatible
    api_base: http://homebox.tail1234.ts.net:8000
    discovery: dynamic
    credential: none

  bedrock-projA-gov:             # 2 accounts × {gov, com} = 4 entries, 4 credential names
    type: bedrock
    region: us-gov-west-1
    credential: bedrock-projA-gov
    models: [us-gov.anthropic.claude-sonnet-5]  # static, curated
  bedrock-projA-com:
    type: bedrock
    region: us-east-1
    credential: bedrock-projA-com
    models: [us.anthropic.claude-sonnet-5]
# bedrock-projB-gov, bedrock-projB-com: same shape

  gemini:
    type: gemini
    credential: gemini-api-key
    models: [gemini-2.5-pro, gemini-2.5-flash]
```

- `discovery: dynamic` vs. a static `models: list` is per-backend. vLLM churns under other people's administration, so probe it; Bedrock's catalog is stable and huge, so curate the entries actually used. A backend can have both: the static list is the fallback when the probe fails.
- `type` selects an adapter: how to probe, and how to render `litellm_params`. One `openai-compatible` adapter covers vLLM, LiteLLM peers, and anything else speaking `/v1/models` — home vLLM and N work vLLM instances are the same code. Ollama gets its own small adapter (`/api/tags`, and `/api/show` for extra facts) as a home convenience.
- Sensitive addresses may omit `api_base` and defer to `machine.yaml` — the registry entry then documents *that* the backend exists, not *where*.

Credential store — private, per machine

```
~/.config/multiagent/credentials/
gemini-api-key.env          # GEMINI_API_KEY=...
```

```

bedrock-projA-gov.env      # AWS_ACCESS_KEY_ID=..., short-term, rewritten each morning
bedrock-projA-com.env
bedrock-projB-gov.env
bedrock-projB-com.env

```

One file per credential name, KEY=value lines, mode 0600. Properties:

- **Rotation is free.** Daily short-term AWS keys overwrite a file; registry and projects never change. "Which accounts do I have today" = which files exist and are fresh.
- **Absence is a feature.** A project referencing a missing credential fails at launch with the expected path in the message, and `ma_models` shows that backend as *configured but not credentialed*. A coworker without a Gemini key uses the same registry; that backend is simply dark for them — this status line is their onboarding checklist.
- **Cross-machine sharing is file sync** scoped to one directory — syncthing, scp, or age later, without touching any other component.

projects.yaml — policy, in git

```

projects:
  home:
    backends: [home-vllm, gemini]
  paper-review:
    backends: [home-vllm]           # nothing leaves the house
    default_model: qwen3-30b
  projA-gov:
    backends: [bedrock-projA-gov, work-vllm-3]
  projB:
    backends: [bedrock-projB-com]
    model_filter: [claude-sonnet-5]

```

A project is an *authorization policy*, not a configuration. Without it, "I forgot which account I was on and asked the wrong endpoint" is one muscle-memory slip away. With it, the rendered proxy config for `projA-gov` simply contains no route to any other account or partition — wrong-money models are unreachable, not discouraged. Selecting a project selects credentials, and therefore selects models.

`model_filter` exists because backend granularity isn't always enough: a shared LiteLLM peer (or a big vLLM host) offers what *its owner* chose to host; a project needs the subset *you* choose to use. Availability is the backend's business; authorization is the project's business. Different owners → different components.

models.yaml — belief about models, shared, in git (*new component*)

The problem it solves: a vLLM instance advertises model IDs (and, usefully, `max_model_len`) but not tool-calling support, tokenizer, chat-template behavior, max sane output, or vision capability. Those you guess or ask the administrator — and the answer should be written down *once, for everyone*. This file is where "asked the admin" becomes shared institutional memory, which makes it the most valuable file coworkers contribute to.

Two levels, because facts attach to two different things:

```

models:                                     # facts about the abstract model
  qwen3-30b:
    match: ["qwen3:30b", "Qwen/Qwen3-30B-A3B*"]  # served-id patterns, any backend
    context: 32768
    max_output: 8192
    tools: true
    vision: false
    tokenizer: qwen2

```

```

claude-sonnet-5:
  match: ["*anthropic.claude-sonnet-5*", "claude-sonnet-5*"]
  source: catalog # facts and price defer to the vendored catalog

deployments: # facts about ONE backend's serving of a model
  work-vllm-3:
    "Qwen/Qwen3-30B-A3B":
      max_model_len: 16384 # admin lowered it below the model's native context
      tool_parser: hermes # asked D. Admin, 2026-08-20
      price: {input_per_mtok: 0, output_per_mtok: 0} # internal box, no market price

```

Design decisions embedded here:

- **Canonical names solve the identity problem.** The same model appears as qwen3:30b on ollama, Qwen/Qwen3-30B-A3B on vLLM, and something else on Bedrock. `match:` patterns map served IDs to one canonical entry, and the canonical name becomes the model alias in the rendered proxy config — so agents see stable names (qwen3-30b) no matter which backend serves them, and the agent configuration never changes when infrastructure does.
- **Deployment overrides are separate from model facts** because they have a different owner (the endpoint's administrator) and a different lifetime (they change when the admin redeploys). A deployment override without a matching probed model is itself a signal: the deployment changed and this entry is stale.
- **Provenance goes in comments** (# asked D. Admin, 2026-08-20). A formal provenance field would be ceremony; a dated comment answers “why do we believe this” well enough, and git blame backs it up.
- **Unknown models are usable but flagged.** A probed model with no match is still routed (don't block work on bookkeeping) but `ma models` marks it `no facts` - add to `models.yaml` or ask admin. If that proves too permissive, a per-project `require_facts: true` is a small addition later.

Merge precedence: observation beats belief beats catalog

For each fact about a (backend, served-model) pair:

probed value > deployments override > models entry > catalog entry

- Probes win for anything they can actually observe (existence; vLLM's `max_model_len`; ollama's `/api/show` details). Beliefs win for everything probes can't see (tool support, tokenizer, price).
- **Conflicts are reported, not silently resolved.** If `models.yaml` says context 32768 and the probe says 16384, the merged value is 16384 *and* `ma models` prints the disagreement — a mismatch means beliefs are stale, and the whole point of this file is that someone fixes it once for everyone.

Price catalog — external facts, vendored

Pricing is a *separate lookup with a separate source per backend type*, which is why it is not a field crammed into `backends.yaml`:

- **Hosted APIs (Bedrock, Anthropic, OpenAI, Gemini):** LiteLLM maintains `model_prices_and_context_window.json` a community catalog of per-token prices and context limits keyed by model ID. Vendor a copy in the repo — the work network may not fetch it live, and a pinned copy means everyone computes the same costs. Record its date; refresh deliberately.
- **GovCloud prices differ from commercial**, and the catalog mostly knows commercial. This alone forces a local override layer: a `price:` field on a deployment or model entry beats the catalog (consistent with the precedence rule — an override is a belief correcting an external source).
- **Self-hosted vLLM/ollama have no market price.** Default to zero; a deployment `price:` can assign a nominal internal rate if chargeback ever matters.

Merged prices flow into the rendered proxy config so LiteLLM's spend logging prices each request at request time. Prices change over time, so logged cost is a snapshot — fine for “which project spent what,” which is the actual requirement (“use the right money” is enforced by projects.yaml; pricing just measures it).

machine.yaml — local quirks, private, not policy

Per-machine overrides only: an api_base override (the home box reaches vLLM as localhost; the Mac cannot), proxy port, usage-log directory. Deliberately boring. If policy starts leaking into this file, that's a design smell.

The pipeline

Every launch composes the components in one direction; data flows down and nothing flows back. Probing happens **at agent start, one layer outside the agent, before the proxy comes up** — the same freshness check most agents do internally, done once here so every agent framework benefits and the rendered config reflects reality at launch:

```
backends.yaml    select (project  registry, apply machine overrides)
projects.yaml
machine.yaml

credential store  resolve names → env vars          (fail loud if missing)

per-backend probes  discover live models          (vLLM/LiteLLM /v1/models,
                                                    ollama /api/tags)
                  diff vs last-seen snapshot → report changes,
                  then overwrite snapshot

models.yaml + catalog  merge facts per (backend, model)
                      warn: conflicts, unknown models, stale overrides

render litellm config  routes under canonical names, model_info + prices,
                      secrets as os.environ/VAR references, NOT values

launch  proxy (container sidecar, in-container, or plain process)
```

agent

ma models --project X is the pipeline stopped after the merge, printed:

backend	credential	status	model (canonical)	ctx	tools	price	note
work-vllm-3	(none)	live	qwen3-30b	16384*	yes	internal	*probe < belie
bedrock-projA-gov	bedrock-projA-gov	ok	claude-sonnet-5	200k	yes	catalog(gov override)	
bedrock-projB-com	bedrock-projB-com	NO KEY	-				
home-vllm	(none)	CHANGED	llama-3.3-70b	128k	yes	internal	was qwen3-30b

The snapshot exists only to produce the CHANGED line and is overwritten after each report. It is never an input to selection or rendering — a cache used for routing lies, but a diary used for diffing doesn't. Whether anything polls in the background remains open; launch-time diff is the minimal version and may be the permanent one.

Security analysis

Threat model, in decreasing order of likelihood:

1. **A key gets committed to git.** Classic, and this directory is currently one `git init && git add .` away from it: `./.env` holds real keys copied from another project — copied `.env` files are how keys spread. Design answer: secrets live *only* under `~/.config/multiagent/credentials/`, which no git repo contains. Every shared file — registry, projects, model knowledge, catalog — is safe to publish *by construction*, not by discipline. Immediate chores: move the keys out of `./.env`, gitignore `.env` before the first commit anyway.
2. **The agent reads the key.** The agent's job is running tools with filesystem access, and its context is fed to a model — a prompt-injected or curious agent that can read a key may exfiltrate it. Consequences:
 - **The workspace mount is the agent's readable surface.** Credentials must live outside every directory ever mounted as a workspace.
 - **The proxy holds the secrets, not the agent.** The entrypoint starts LiteLLM with provider env vars, then scrubs them before exec-ing the agent, which receives exactly one secret: a per-launch random LiteLLM master key valid only against `localhost:4000`, worthless outside, dead at session end. Caveat: same-UID processes in one container mean a determined agent could read `/proc//environ` — a raised bar, not a wall. The wall is a separate proxy container or process; the security argument and the federation argument push toward the same shape.
3. **Shared files leak infrastructure details.** Internal hostnames, project names hinting at employers or contracts. Mitigations: `api_base` deferral to `machine.yaml`, and splitting `projects.yaml` (or `models.yaml`'s `deployments` section) into a private repo — the schema doesn't care where each file comes from.
4. **Rendered configs and logs.** Configs reference `os.environ/VAR`, never values, so they're safe to keep, diff, and attach to bug reports. Usage JSONL records tokens, models, and computed cost — never prompts, never keys. Boring artifacts by default.

Principle: **secrets flow through exactly one narrow path** (credential file → env-file → proxy process env); everything else is clean by construction. Widening that path should feel like a design change, not a convenience.

The shared-endpoint case (coworker's or friend's LiteLLM/vLLM)

Someone offers a hosted endpoint with its own model list, usage accounting, and virtual keys. What changes here? One registry entry (`type: openai-compat, discovery: dynamic`), one credential file, and — the instructive part — project policy still matters even though the backend does its own tracking: their endpoint offers what *they* host; your project uses the subset *you* authorize. Their forty models, your three. Availability is theirs; authorization is yours; `models.yaml` beliefs about their models are *shared between you*, because the facts problem (“does it do tools?”) is the same on their box as on yours. Your local proxy chains to theirs — a few milliseconds for uniform logging, uniform canonical names, and one unchanging agent configuration. The symmetric case is free: your proxy can issue virtual keys to coworkers, and the registry format is already the interchange format.

Deployment topology and the portability constraint

The work network may not pull from PyPI, ghcr, or Docker Hub, and Docker itself may not be permitted there. So the container is a **packaging choice, not an architectural requirement**:

1. **Now (home):** proxy inside the agent container. Zero port management, per-launch config snapshot, one `docker run`.
2. **When Bedrock keys are in play (work):** proxy as a sidecar container on a per-launch network — the agent container never receives provider secrets. Gov credentials deserve the wall, not the raised bar.
3. **No Docker available:** proxy as a plain local process, or — weakest — skip the proxy and export env vars directly to the agent, accepting the loss of secret isolation, canonical names, and uniform logging, and saying so loudly at launch.
4. **Later:** one long-lived proxy per machine; remote proxies as peers.

Each step changes only where the rendered config is applied and what base URL agents get. Registry, credentials, projects, model knowledge, and discovery are untouched — which is the main evidence the decomposition is right. The launcher itself stays near stdlib + YAML so it installs anywhere.

Implementation order

- The merge stage (probe × models.yaml × catalog, with precedence and conflict reporting) is most of the “complex interaction” this project exists for; build and test it as a pure function from inputs to a merged table, independent of any proxy or container.
- Two backends from day one, chosen to exercise everything: `home-vllm` (dynamic discovery, deployment overrides, zero price) and one hosted API (static list, credential resolution, catalog pricing). Bedrock is then a new adapter plus four credential files, not a redesign.
- The entrypoint scrubs provider env vars before exec-ing the agent; the render stage emits `model_info` and price fields alongside routes.
- Immediate chores: `git init` with `.gitignore` covering `.env`; move keys to `~/.config/multiagent/credentials/`.

Open questions

- Do models.yaml deployments: overrides stay in models.yaml (all beliefs in one file; one file for coworkers to edit) or move under the backend entries in backends.yaml (facts next to the endpoint they describe)? Current choice: models.yaml, keeping backends.yaml pure connectivity. Revisit if the two files' edit patterns turn out to correlate.
- Background polling vs. launch-time diff for change monitoring. Launch-time is the minimal version; add polling only if a real need appears.
- Should short-term credential files trigger a staleness warning after N hours? Cheap to add alongside `ma keys`; pointless before.
- Where projects.yaml (and sensitive deployment names) live once names get sensitive — same repo, or a private overlay repo. Schema is indifferent.
- Vocabulary, fixed from the first commit: *backend* (a reachable thing you send tokens to), *credential* (the named secret it requires), *project* (the policy joining them), *model* (the abstract thing facts attach to), *deployment* (one backend's serving of one model).